

架构候选方案对比表

团队名称：哪一队 日期：2026-04-28

文档版本：V1.0

一、项目背景

本项目为"Campus Helper"校园互助服务平台，基于 Phase 1 需求分析结果，系统需支持以下核心功能：

- 用户实名认证（手机号+学生证审核）
- 任务发布与接单（快递代取、学习辅导、二手交易、组队匹配等）
- 订单状态流转与托管支付
- 信用评价与互评体系
- 管理后台（审核、纠纷处理、数据看板）

关键约束条件：

约束项	具体说明
团队规模	4人（大学生团队）
开发周期	10周（Phase 2 为第8-9周，后续还有编码、测试阶段）
技术储备	熟悉java语言
目标用户	单校大学生，初期并发用户 ≤ 3000
部署环境	低成本云服务器或免费 PaaS 平台
预算限制	学生项目，无商业预算

二、候选方案说明

方案A：分层架构（Layered Architecture）

采用经典四层分层架构，前后端分离，后端按职责水平切分为表示层、业务层、数据访问层和基础设施层。系统以单体应用形式部署，通过模块化代码组织实现关注点分离。

层级	技术/组件
前端层	Vue 3 微信小程序 / 管理后台 Web

层级	技术/组件
接口层	RESTful API / 统一响应 / JWT 鉴权
业务层	用户、任务、订单、支付、信用、消息模块
数据层	MyBatis-Plus / MySQL / Redis
基础设施层	文件存储、日志、监控、消息推送

方案B：微服务架构（Microservices Architecture）

将系统拆分为多个独立部署的服务，每个服务围绕业务能力构建，拥有独立数据库，通过 RESTful API 或消息队列通信，使用服务注册发现、配置中心、网关等基础设施。

列1	列2	列3	列4
API Gateway (网关)	API Gateway (网关)	API Gateway (网关)	API Gateway (网关)
用户服务 (Auth)	任务服务 (Task)	订单服务 (Order)	支付服务 (Payment)
服务注册发现 (Consul/Nacos)	服务注册发现 (Consul/Nacos)	服务注册发现 (Consul/Nacos)	服务注册发现 (Consul/Nacos)
用户DB (MySQL)	任务DB (MySQL)	订单DB (MySQL)	支付DB (MySQL)

三、多维度对比分析

对比维度	方案A：分层架构	方案B：微服务架构
开发复杂度	★★ 低 单体应用，模块间通过函数调用通信，调试简单，IDE 支持完善	★★★★★ 极高 需处理分布式事务、服务间通信、数据一致性、服务治理等问题，调试需跨多个服务
可维护性	★★★ 良好 代码集中，重构方便；但随着代码量增长，可能出现"大泥球"风险	★★★ 一般 服务边界清晰，单服务易维护；但跨服务变更需协调多个代码库，运维复杂度高
可扩展性	★★★ 够用 可通过单体多实例部署应对并发增长；数据库可读写分离	★★★★★ 优秀 可针对热点服务独立扩容（如支付服务）；技术栈可按服务独立选型
性能	★★★★★ 良好 进程内调用无网络开销，延迟低；满足 ≤3000 并发需求绰绰有余	★★★ 一般 引入网络通信开销（RPC/HTTP）；分布式事务进一步降低性能；对当前规模属过度设计

对比维度	方案A：分层架构	方案B：微服务架构
团队学习成本	☆☆☆ 良好 需要学习Spring Boot 单体开发相关，需额外学习分布式技术栈	☆☆☆☆☆ 极高 需学习服务注册发现、熔断限流、分布式事务（Seata）、容器编排（K8s）等，10周内难以掌握
10周内可行性	☆☆☆☆☆ 完全可行 可专注业务功能实现，风险可控	☆☆ 风险极高 基础设施搭建需2-3周，剩余时间难以完成核心业务功能；团队无运维经验，生产环境稳定性无保障
部署成本	☆☆ 低 单台云服务器即可运行，Docker 单容器部署，运维简单	☆☆☆☆☆ 高 需多台服务器或 K8s 集群，配置中心、监控、日志收集等基础设施均需资源
测试难度	☆☆ 低 集成测试在单进程内完成，测试数据准备简单	☆☆☆☆☆ 高 需搭建服务依赖环境，契约测试、端到端测试复杂度高
数据一致性	☆☆☆☆☆ 极易保障 单机数据库事务（ACID）天然支持	☆☆ 复杂 分布式事务需引入 Saga/TCC 等模式，开发成本高，易出现不一致

四、关键场景分析

场景1：订单状态流转（核心业务流程）

订单从"发布中 → 已接单 → 进行中 → 待确认 → 已完成"的流转涉及任务、订单、支付、信用四个模块。

方案	实现方式	风险
分层架构	同一事务内完成所有状态变更，数据库 ACID 保障一致性	无
微服务	需跨服务协调（订单服务 + 支付服务 + 信用服务），引入分布式事务	高：10周内难以正确实现，易出现数据不一致

场景2：MVP 快速验证

项目需在 10 周内完成可演示的系统，并预留时间进行测试和修复。

方案	启动到首版可用时间	风险
分层架构	1-2 周完成基础框架，第 3 周开始业务功能	低

方案	启动到首版可用时间	风险
微服务	2-3 周搭建基础设施（网关、注册中心、配置中心），第 4 周才能开始业务功能	高：时间紧迫，功能可能无法完整交付

场景3：团队人员变动或任务阻塞

4人团队中若有人因考试/生病暂时离开。

方案	影响	应对
分层架构	其他成员可快速接手任何模块（代码在同一仓库）	容易
微服务	若负责支付服务的成员离开，其他人需理解其服务边界和技术选型	困难

五、综合评分

评估维度	权重	方案A得分	方案B得分
开发复杂度	20%	9	3
可维护性	15%	7	6
可扩展性	10%	6	9
性能	10%	8	5
团队学习成本	20%	7	3
10周内可行性	25%	10	3
加权总分	100%	8.15	4.25

评分标准：10分制，10分为最优

六、初步结论

基于以上分析，团队初步倾向选择方案A（分层架构），核心原因：

- 1. 时间约束：10周周期内，分层架构可确保核心业务功能完整交付，微服务架构的基础设施搭建将严重挤压业务开发时间。
- 2. 需求规模匹配：单校 ≤3000 并发用户，分层架构通过单体多实例部署完全可满足性能需求，微服务的扩展性优势在此场景下无法体现。
- 3. 风险可控：分层架构的调试、测试、部署均简单可控，适合学生项目的运维能力。

微服务架构的适用时机：若未来平台扩展至多校运营、用户量突破万级、团队具备 DevOps 能力时，可考虑从分层架构向微服务演进（如将支付、通知服务率先拆分）。

文档编制：LXY

审核状态：已经由人工审查